Voice Interview 项目架构技术明细

> *生成日期: 2026-03-28*

一、整体架构

```
voice-interview/
├── voice-interview-mobile/     # 移动端 H5 + 小程序
├── voice-interview-admin/      # 管理后台
└── voice-interview-backend/    # Java 后端服务
```

三端独立工程，通过 REST API 和 WebSocket 通信。后端统一提供认证、面试引擎、媒体管理、题库管理等服务。

二、移动端（voice-interview-mobile）

2.1 技术栈

分类	技术	版本
框架	uni-app	3.0.0-4080420251103001
UI 框架	Vue 3	^3.4.21
语言	TypeScript	^5.4
状态管理	Pinia	2.1.7
国际化	vue-i18n	^9.1.9
构建工具	Vite	5.2.8
多端支持	H5 / 微信小程序 / 支付宝 / 百度	-

2.2 页面结构

页面	路径	功能
登录/注册	pages/login/index	JWT 登录、注册、持久化
首页	pages/index/index	面试入口、快速开始
面试配置	pages/interview/setup	预设题库选择 / 简历上传匹配
实时面试	pages/interview/session	WebSocket + 语音问答
面试报告	pages/interview/report	评分、建议、音频回放
历史记录	pages/history/list	历史面试列表

2.3 核心 Composables

文件	功能
useMediaUpload.ts	录音 + 上传，原生/浏览器双路兼容
useAudioPlayback.ts	音频回放控制
useInterviewSocket.ts	WebSocket 连接、重连、消息派发
useWaveform.ts	64 柱声波历史管理，RMS 振幅计算，requestAnimationFrame 驱动

2.4 语音录制方案

- **H5**：Web Audio API `ScriptProcessorNode` 捕获 Float32Array，编码为 WAV 上传
- **小程序**：`uni.getRecorderManager()` 原生录音
- **声波可视化链路**：`onaudioprocess` → `onAudioSamples` 回调 → `useWaveform.pushSamples()` → Canvas 实时绘制 64 柱波形

三、管理后台（voice-interview-admin）

3.1 技术栈

分类	技术	版本
框架	Vue 3	^3.5
UI 组件库	Ant Design Vue	^4.2
路由	Vue Router	^4.4
状态管理	Pinia	^2.2
构建工具	Vite	^8.0
语言	TypeScript	^5.6

3.2 页面功能

页面	功能
登录	用户名/密码登录，固定后端地址
分类管理	题库分类 CRUD、树形结构
题目管理	题目增删改查、批量导入
历史记录	查看所有用户面试历史
报告查看	面试评分报告详情
系统监控	媒体文件清理、健康检查

四、后端服务（voice-interview-backend）

4.1 基础框架

分类	技术	版本
框架	Spring Boot	3.5.7
语言	Java	21
构建	Maven	3.9+
ORM	MyBatis-Plus	3.5.9
数据库	MySQL	8.0+（阿里云 RDS）
缓存	Redis	可选，默认关闭
JSON	Jackson	Spring Boot 内置
验证	Spring Validation	Spring Boot 内置
任务调度	Spring @Scheduled	媒体文件定时清理

4.2 模块结构

模块	包路径	职责
用户认证	module/user	注册、登录、JWT、用户信息
面试引擎	module/interview	会话状态机、WebSocket、报告生成
题库管理	module/library	分类、题目 CRUD、文本导入
AI 服务	module/ai	LLM 对话，OpenAI 兼容接口
ASR 服务	module/asr	语音识别，多 Provider 抽象
TTS 服务	module/tts	语音合成，多 Provider 抽象
媒体管理	module/media	文件上传、存储、生命周期
简历解析	module/interview/resume	PDF 解析、面试题生成
系统管理	module/system	健康检查、媒体清理、TTS 测试

4.3 认证方案

- **方式**：JWT Bearer Token
- **签发**：登录成功后返回，有效期 7 天（604800 秒）
- **WebSocket**：ticket 机制，先 HTTP 换取一次性 ticket，再 WS 连接携带，有效期 300 秒
- **Provider**：`mock`（开发默认）/ `jdbc`（真实数据库）

4.4 面试引擎

```
客户端 --HTTP--> POST /api/interview/start  --> 创建会话
客户端 --HTTP--> GET  /api/interview/ws-ticket --> 换取 WS ticket
客户端 --WS---> /ws/interview?ticket=xxx     --> 建立实时通道
客户端 --> [发送音频文件ID / 文本] --> ASR转写 --> LLM生成回复 --> TTS合成 --> 推送消息
客户端 --HTTP--> POST /api/interview/end      --> 生成报告
```

会话存储 Provider:

Provider	条件	说明
memory	默认	内存存储，重启丢失
redis	app.interview.session-store=redis	Redis 持久化
jdbc	app.interview.session-store=jdbc	MySQL 持久化

4.5 AI Provider 抽象层

LLM（大语言模型）

Provider	配置值	说明
mock	app.ai.provider=mock	固定回复，开发用
openai	app.ai.provider=openai	OpenAI 兼容接口，当前用 gpt-5.4 via dejong21.me

ASR（语音识别）

Provider	配置值	说明
mock	app.asr.provider=mock	固定转写文本
openai	app.asr.provider=openai	OpenAI Whisper 兼容
dashscope	app.asr.provider=dashscope	阿里云百炼 qwen3-asr-flash-realtime
sherpa	app.asr.provider=sherpa	本地离线 Sherpa-ONNX

TTS（语音合成）

Provider	配置值	说明
mock	app.tts.provider=mock	返回空音频
openai	app.tts.provider=openai	OpenAI TTS 兼容
dashscope	app.tts.provider=dashscope	阿里云百炼 qwen3-tts-flash-realtime，声音 Cherry
mimo	app.tts.provider=mimo	小米 Mimo TTS
sherpa	app.tts.provider=sherpa	本地离线

4.6 数据库设计

表名	说明
t_user	用户账号
t_category	题库分类（树形）
t_question	面试题目
t_import_task	题目导入任务
t_media_file	媒体文件记录
t_interview_session	面试会话
t_interview_question	会话快照题目
t_interview_round	每轮对话记录
t_interview_report	面试报告（JSON）

所有表使用逻辑删除（`deleted` 字段），MyBatis-Plus 自动过滤。

4.7 媒体存储

- **存储方式**：本地文件系统（`./storage/media`）
- **支持格式**：WAV、MP3、M4A、WebM、OGG 等
- **清理机制**：`@Scheduled` 定时任务扫描过期媒体文件，配合 `expire_at` 字段
- **访问方式**：通过 `/api/media/{fileKey}` 代理返回文件流

4.8 简历解析

- **依赖**：Apache PDFBox 2.0.31
- **流程**：上传 PDF → 提取文本 → 调用 LLM 生成定制面试题 → 返回题目列表

五、当前运行配置

项目	地址
移动端 H5	http://localhost:5174
管理后台	http://localhost:5173
后端服务	http://localhost:8080
数据库	阿里云 MySQL 47.116.36.30:3306/voice-interview
LLM	https://api.dejong21.me/v1，模型 gpt-5.4
ASR/TTS	阿里云百炼 DashScope，qwen3-asr/tts-flash-realtime

Spring Profiles 激活顺序

```
dev → openai → dashscope → local
```

后加载的 profile 覆盖前面的同名配置项。

六、启动方式

```
# 后端
cd voice-interview-backend
mvn spring-boot:run

# 移动端
cd voice-interview-mobile
npm install --legacy-peer-deps
npm run dev:h5

# 管理后台
cd voice-interview-admin
npm install
npm run dev
```